Vector Programs contd..

***What is Fail Fast Ieartor ?

[Modification is only allowed through Iterators own methods]

While iterating (retrieving) the object from the collection by using Itearor interface or for each loop (java.lang.Iterable) OR ListIterator interface, if at any point of time the original Collection structure is going to modify after the creation of Itearator by using any way except Iterators's own method [remove(), add(), set()] then It will throw an exception i.e. **java.util.ConcurrentModificationExacption.**

```java
package com.ravi.vector_demo;

import java.util.Vector;

record Employee(Integer id, String name, Double salary)
{

}

public class VectorDemo5
{
    public static void main(String[] args)
    {
        Vector<Employee> listOfEmployees = new Vector<>();
        listOfEmployees.add(new Employee(101, "Scott", 78890.90));
        listOfEmployees.add(new Employee(102, "Smith", 98890.90));
        listOfEmployees.add(new Employee(103, "John", 65890.90));
        listOfEmployees.add(new Employee(104, "Alen", 91890.90));
        listOfEmployees.add(new Employee(105, "Sanju", 99890.90));

        IO.println("Original Object");
        for(Employee employee : listOfEmployees)
        {
            IO.println(employee);
        }

         Integer id = Integer.valueOf(IO.readln("Enter the employee id to delete : "));

         for(Employee employee : listOfEmployees)
        {
            if(employee.id().equals(id))
            {
              listOfEmployees.remove(employee);
              break;  //break is compulsory
            }
        }

         IO.println("Remaining Objects after deleting :");
        for(Employee employee : listOfEmployees)
        {
            IO.println(employee);
        }
    }
}
```

Note : In the above program break is compulsory otherwise we will get java.util.ConcurrentModificationException but even after using break, we can delete only one object so, this technique is not suitable for multiple deletion of the object.
In order to delete multiple objects we should use Iterator -> remove() method as shown below

```java
package com.ravi.vector_demo;

import java.util.Iterator;
import java.util.List;
import java.util.Vector;

record Player(Integer id, String name, Double basePrice)
{

}


class IPLTeam
{
    private String name;
    private List<Player> listOfPlayer;


    public IPLTeam(String name)
    {
        this.name = name;
        listOfPlayer = new Vector<>(15);
    }

    public void addPlayer(Player player)
    {
      listOfPlayer.add(player);
    }

    public void showListOfPlayer()
    {
        listOfPlayer.forEach(IO::println);
    }

    public void removeOrRetain()
    {
        Iterator<Player> iterator = listOfPlayer.iterator();

        while(iterator.hasNext())
        {
            Player player = iterator.next();

            if(player.basePrice() >= 5000)
            {
                iterator.remove();
            }
        }

    }


}

public class VectorDemo6
{
    public static void main(String[] args)
    {
        IPLTeam team = new IPLTeam("SRH");
        team.addPlayer(new Player(101, "Abhishek", 4500D));
        team.addPlayer(new Player(102, "Ishan", 4900D));
        team.addPlayer(new Player(103, "Cummines", 5900D));
        team.addPlayer(new Player(104, "HEAD", 5500D));
        team.addPlayer(new Player(105, "Nitish", 4800D));
        team.showListOfPlayer();
        team.removeOrRetain();
        IO.println("Final player List :");
        team.showListOfPlayer();
    }

}
```

```java
package com.ravi.vector_demo;

import java.util.List;
import java.util.Vector;

//Fetch all the premium customer whose bill is greater then 1 lakh

record Customer(Integer id, String name, Double bill)
{

}

public class VectorDemo7
{
    public static void main(String[] args)
    {
        Customer c1 = new Customer(101, "Scott", 120000D);
        Customer c2 = new Customer(102, "Smith", 90000D);
        Customer c3 = new Customer(103, "Raj", 150000D);
        Customer c4 = new Customer(104, "Alen", 95000D);
        Customer c5 = new Customer(105, "John", 125000D);
        Customer c6 = new Customer(106, "Black", 78000D);
        Customer c7 = new Customer(107, "Sanju", 160000D);

        Vector<Customer> listOfCustomers = new Vector<>();
        listOfCustomers.add(c1);
        listOfCustomers.add(c2);
        listOfCustomers.add(c3);
        listOfCustomers.add(c4);
        listOfCustomers.add(c5);
        listOfCustomers.add(c6);
        listOfCustomers.add(c7);

        IO.println("Original Customer List :");
        for(Customer customer : listOfCustomers)
        {
            IO.println(customer);
        }

        //Approach 1
        IO.println("Premimum Customers :");

         List<Customer> premimumCustomers = new Vector<>();

        for(Customer customer : listOfCustomers)
        {
          if(customer.bill() >=100000)
          {
              premimumCustomers.add(customer);
          }
        }
        premimumCustomers.forEach(IO::println);

    }

}
```

```java
package com.ravi.vector_demo;

import java.util.Vector;

//Updating the Student marks based of the student id and sort student objects by name

record Student(Integer id, String name, Double marks)
{

}

public class VectorDemo8
{
    public static void main(String[] args)
    {
        Vector<Student> listOfStudents = new Vector<>();
        listOfStudents.add(new Student(333,"Aryan",90D));
        listOfStudents.add(new Student(111,"Zuber", 85D));
        listOfStudents.add(new Student(222,"Raj",64D));

        IO.println("Before Updation of the Marks :");
        listOfStudents.forEach(IO::println);

        Integer id = Integer.valueOf(IO.readln("Enter the id of the student :"));
        Double newMarks = Double.valueOf(IO.readln("Enter the updated marks student :"));

        boolean isFound = false;

        for(int i=0; i<listOfStudents.size(); i++)
        {
            if(listOfStudents.get(i).id().equals(id))
            {
                listOfStudents.set(i, new Student(id, listOfStudents.get(i).name(), newMarks));
                isFound = true;
                break;
            }
        }

        if(!isFound)
        {
            IO.println("Sorry "+id+" is not existing");
        }

        IO.println("After Updation of the Marks :");
        listOfStudents.forEach(IO::println);

        IO.println("Sorting the student object based on the name :");
        listOfStudents.sort((s1,s2)-> s1.name().compareTo(s2.name()));

        IO.println("After Sorting based on the name :");
        listOfStudents.forEach(IO::println);

    }

}
```

---

default Iterator<E> asIterator() :

* This default method **asIterator()** is added inside **Enumeration<E> interafce** from JDK 9V.

* It will convert the **Enumeration to behave as Iterator<E>**

```java
package com.ravi.vector_demo;

import java.util.Enumeration;
import java.util.Iterator;
import java.util.Vector;

//sort + asIterator()

record Trainer(Integer id, String name)
{

}

public class VectorDemo9 {

    public static void main(String[] args)
    {
        Vector<Trainer> listOfTrainers = new Vector<>();
        listOfTrainers.add(new Trainer(111, "Scott"));
        listOfTrainers.add(new Trainer(222, "Smith"));
        listOfTrainers.add(new Trainer(333, "John"));
        listOfTrainers.add(new Trainer(444, "Alen"));

        IO.println("Original Object :");
        listOfTrainers.forEach(IO::println);

        IO.println("Sorting based on the name :");
        listOfTrainers.sort((t1,t2)-> t1.name().compareTo(t2.name()));

        IO.println("After Sorting the Trainer Object based on the name :");
        listOfTrainers.forEach(IO::println);

        IO.println("Enumeration as Iterator");
        Enumeration<Trainer> elements = listOfTrainers.elements();
        Iterator<Trainer> itr = elements.asIterator();
        itr.forEachRemaining(IO::println);

    }

}
```

Fixed langth Array & Immutable List :